Ex-6

Error Detection & Correction using Hamming Code

Aim:

To write a program to implement error detection & correction using the Hamming code.

Algorithm:

1) Read the binary data bit
2) Count the no of data bits m.
3) find the number of parity bits r using $2^r \geq m+r+1$
4) create positions for Hamming code
5) place parity bits at position 1, 2, 4, 8 ...
6) Calculate each parity bit using XOR
7) Display the Hamming code
8) Read code and check the parity bits
10) If error position is not zero, flip the bit to correct the error.

Code:

```
def calculate_parity_bits(data):
    m = len(data)
    r = 0
    while (2**r) < (m+r+1):
        r += 1
    return r

def generate_hamming_code(data):
    m = len(data)
    r = calculate_parity_bits(data)
    n = m + r
    hamming = ['0'] * (n+1)
    j = 0
    for i in range(1, n+1):
        if i & (i-1) != 0:
            hamming[i] = data[j]
            j += 1
```

```
    for p in range(r):
        position = 2**p
        parity = 0
        for i in range(1, n+1):
            if i & position:
                parity ^= int(hamming[i])
        hamming[position] = str(parity)
    return ''.join(hamming[1:])

def detect_and_correct(received):
    n = len(received)
    r = 0
    while (2**r) < (n+1):
        r += 1
    hamming = ['0'] + list(received)
    error_position = 0
    if error_position != 0:
        print("Error position", error_position)
        if hamming[error_position] == '0':
            hamming[error_position] = '1'
        else:
            hamming[error_position] = '0'
    return ''.join(hamming[1:])
```

output:

Enter the binary data : 1011

original Data : 1011

Hamming Code : 011 0011

Enter the recived Hamming code : 011 0111

Error position : 5

Error deleted at position 5

Error Corrected Succesfully !

Recived Code : 0110111

Corrected Code : 0110011

Result:

The Hamming Code program was Succesfully